॥ श्रीगणेशाय नमः ॥

Heaps

	Insert	Search	Find Min	Delete Min
Unsorted Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Sorted Array	$O(n)$	$O(logn)$	$O(1)$	$O(n)$
Unsorted Linked List	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Min Heap	$O(logn)$	$O(n)$	$O(1)$	$O(logn)$

```
| Data Structure       | Insert         | Search     | Find Min  | Delete Min |
|----------------------|----------------|------------|-----------|------------|
| Unsorted Array       | O(1)           | O(n)       | O(n)      | O(n)       |
| Sorted Array         | O(n)           | O(log n)   | O(1)      | O(n)       |
| Unsorted Linked List | O(1)           | O(n)       | O(n)      | O(n)       |
| Min Heap             | O(log n)       | O(n)       | O(1)      | O(log n)   |
```

A heap is a priority queue. E.g. max heap: prioritizes maximum elements.

Complete binary tree

All leaves are at the same level.
All non-leaves have 2 children.



Almost complete binary tree

All leaves are at the last or at the level above it.
It should be filled left-to-right.

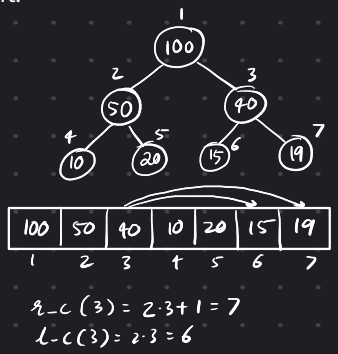


Max heaps

Max heap property: every node is greater than or equal to all the nodes in its sub-tree.
Even though a max heap is interpreted as a tree, it is represented as an array.

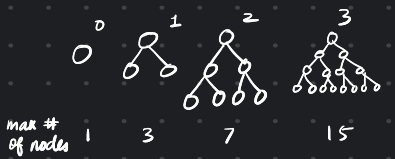
Adv of complete binary tree:
If we represent them as arrays, there'll be no gaps.
A heap is a complete/almost complete binary tree which follows the heap property.

left_child(i) = 2i
right_child(i) = 2i+1
parent(i) = $\lfloor i/2 \rfloor$



Height of node: length of the longest # of edges to a leaf.
Height of tree: height of root node.

If the height of a complete binary tree is h, then the max # of nodes in the tree is $2^{h+1} - 1$.



If there are n nodes in an almost complete binary tree, then height of the tree is $\lfloor \log_2 n \rfloor$

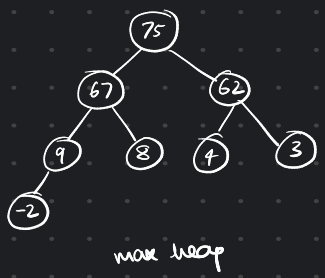
To create max heap

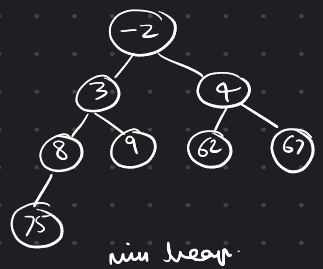
9 8 75 3 4 67 -2 62

Sort in descending order

75 67 62 9 8 4 3 -2

Sort in increasing order to get a min heap



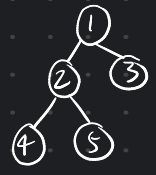


But that takes O(nlogn) time. There's another way to build a heap that takes O(n) time.

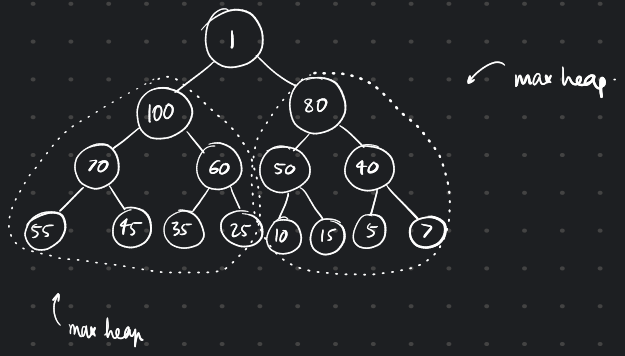
Observation:
Every leaf itself is a heap.
We only need to focus on non-leafs. We need to know where leaves stop and non-leaves begin.
In an almost complete binary tree,

the leaves are $(\lfloor \frac{n}{2} \rfloor + 1)$ to n

the non-leaves are 1 to $\lfloor \frac{n}{2} \rfloor$



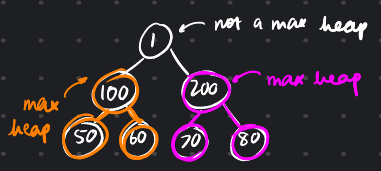
3,4,5 - leaves
1,2 - non leaves

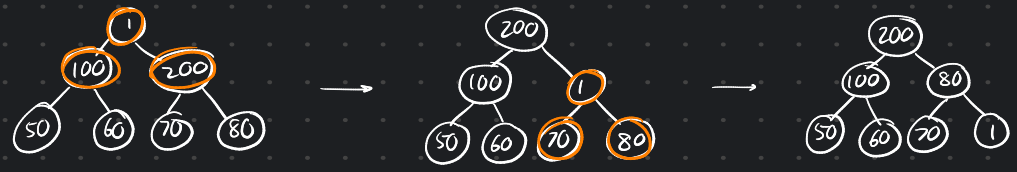


If the left and right subtrees are max heaps,
but the root prevents the whole tree from being one,
then heapify can be used.

max heap

Consider an example:





compare 3 elements
swap the root with the greater.

compare 3 elements.
swap the largest.

A max heap need not be sorted, but any sorted array is a min heap/ max heap.

Heapify

```python
def max_heapify(a, i):
    l = 2*i
    r = 2*i + 1

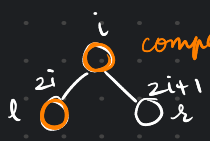
    if l <= a.heap_size and a[l] > a[i]:
        largest = l
    else:
        largest = i

    if r <= a.heap_size and a[r] > a[largest]:
        largest = r

    if largest != i:
        a[i], a[largest] = a[largest], a[i]
        max_heapify(a, largest)
```

if there's a left child (we are not exceeding)

if a right child exists



compare these 2 and get largest

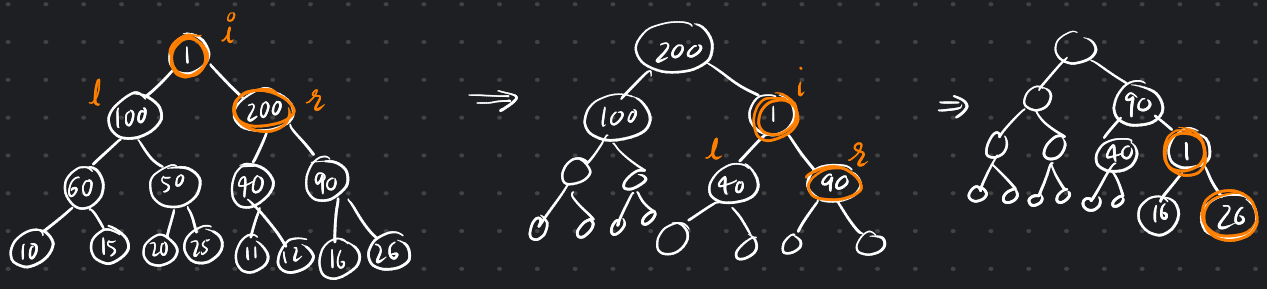


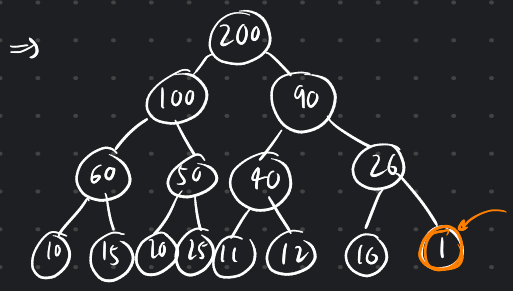
compare this with the previous largest to get the largest of the 3



put largest here

if this one is swapped, apply max-heapify for the subtree below it.



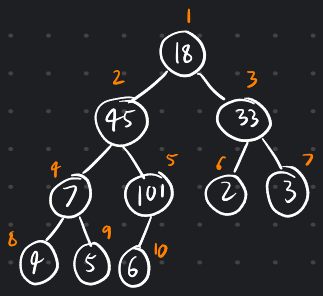


At this point,
largest will not change as there are no left or right children.
max_heapify won't be called further.

Time complexity: $O(logn) = O(h)$
Space complexity: $O(logn)$

Consider an array:

18	45	33	7	101	2	3	4	5	6
1	2	3	4	5	6	7	8	9	10



We apply max_heapify on non-leaves from the end.

5, 4, 3, 2, 1 $\left\lfloor \frac{A.length}{2} \right\rfloor$ to 1

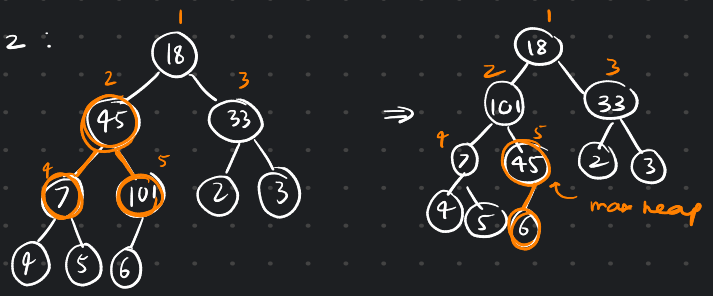
On index 5 : already a heap

4 : heap

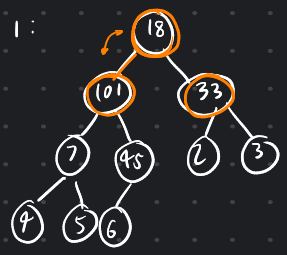
3 : heap

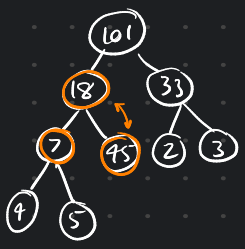
2 : not a max heap

On 2 :



On 1 :



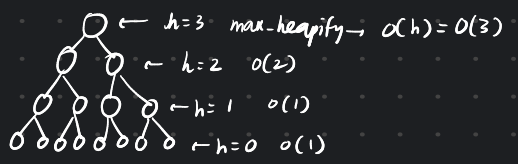




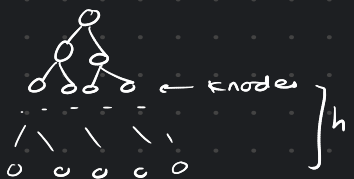
Building a max heap

```
def build_max_heap(a):
    a.heap_size = a.length
    for i in range(a.length//2, 0, -1):
        max_heapify(a, i)
```

Time complexity:



At any level, if height is h, and the level has k elements, then $T = k \cdot O(h)$
Say at the next level l elements are there. $l \times O(h-1)$





of nodes at level h in a complete BT $= \left\lceil \frac{n}{2^{h+1}} \right\rceil$

e.g. at level 0: $\left\lceil \frac{15}{2^{0+1}} \right\rceil = 8$



$\left\lceil \frac{n}{2^{h+1}} \right\rceil * O(h)$ ← for a level h.

$\left\lceil \frac{n}{2^{h+1}} \right\rceil$: nodes at level h; $O(h)$: max_heapify on one node

Total # of levels = $\log n$

h goes from 0 to $\log n$.

At any h, $\left\lceil \frac{n}{2^{h+1}} \right\rceil * O(n)$

$$\text{Total} = \sum_{h=0}^{\log n} \left\lceil \frac{n}{2^{h+1}} \right\rceil * O(h) = \sum_{h=0}^{\log n} \left\lceil \frac{n}{2^h \cdot 2} \right\rceil (c \cdot h)$$

$$= \frac{cn}{2} \sum_{h=0}^{\log n} \frac{h}{2^h}$$

$$\leq \frac{cn}{2} \sum_{h=0}^{\infty} \frac{h}{2^h} \quad \leftarrow 2$$

$$= O(n)$$

total # of levels

of nodes at any level

time for max_heapify on 1 node

time for max_heapify at level h

$$S = \frac{0}{2^0} + \frac{1}{2^1} + \frac{2}{2^2} + \frac{3}{2^3} + \cdots$$

$$\frac{S}{2} = \frac{0}{2^1} + \frac{1}{2^2} + \frac{2}{2^3} + \cdots$$

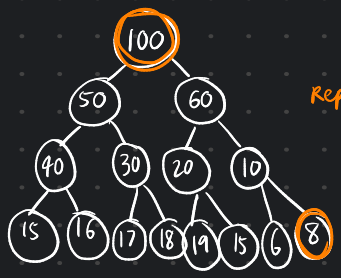
$$\frac{S}{2} = \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots = \frac{1/2}{1-1/2} = 1$$

$$S = 2$$

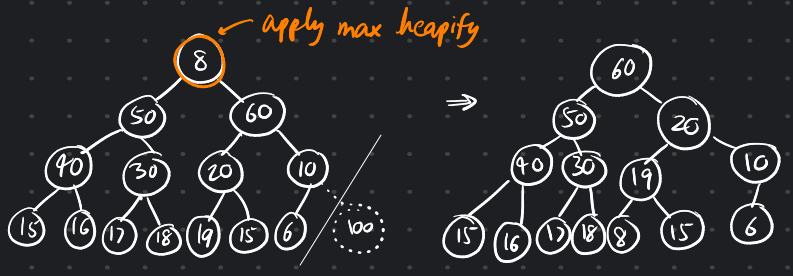
build_max_heap calls max_heapify for one node at a time which takes $O(\log n)$. So a stack of size $O(\log n)$ is needed.

Time complexity for build_max_heap: O(n)
Space complexity for build_max_heap: O(logn)

To delete/extract the root (max) node:



Replace the root with the last node, reduce size by 1 and apply heapify on it.




```python
def heap_extract_max(a):
    if a.heap_size < 1:
        raise ValueError("heap underflow")

    max_element = a[1]
    a[1] = a[a.heap_size]
    a.heap_size -= 1
    max_heapify(a, 1)

    return max_element
```

Time complexity: O(logn)
Space complexity: O(logn)

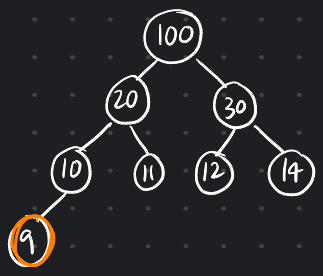
To add nodes one-by-one to an existing heap

Say a heap has $\frac{n}{2}$ nodes:  $n/2$

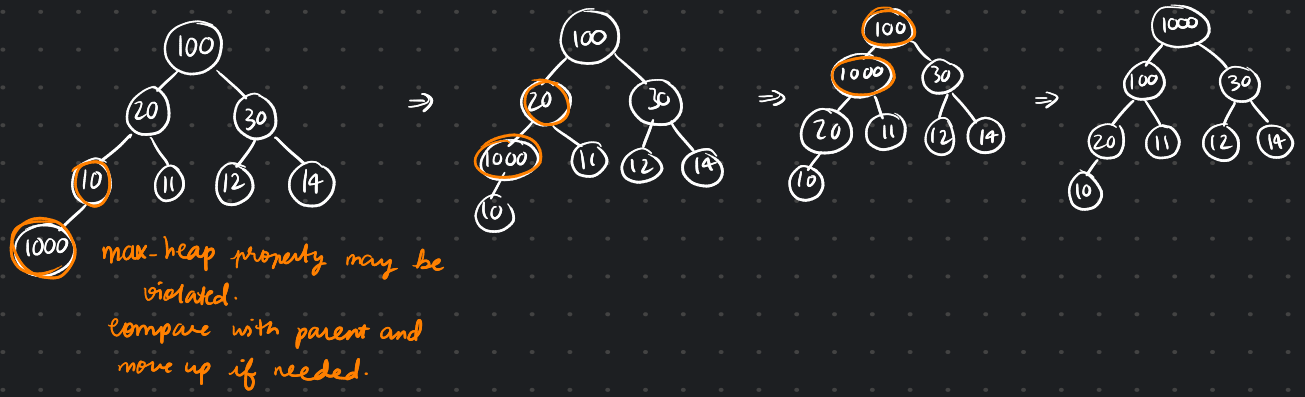
To add $n/2$ more nodes, each would take $O(\log n)$ time.

So $T = O(n \log n)$

To increase the value of a key



Say we want to change 9 → 1000




```
def heap_increase_key(a, i, key):
    if key < a[i]:
        raise ValueError("new key is smaller than current key")
    a[i] = key
    while i > 1 and a[i//2] < a[i]:
        a[i], a[i//2] = a[i//2], a[i]
        i = i//2
```

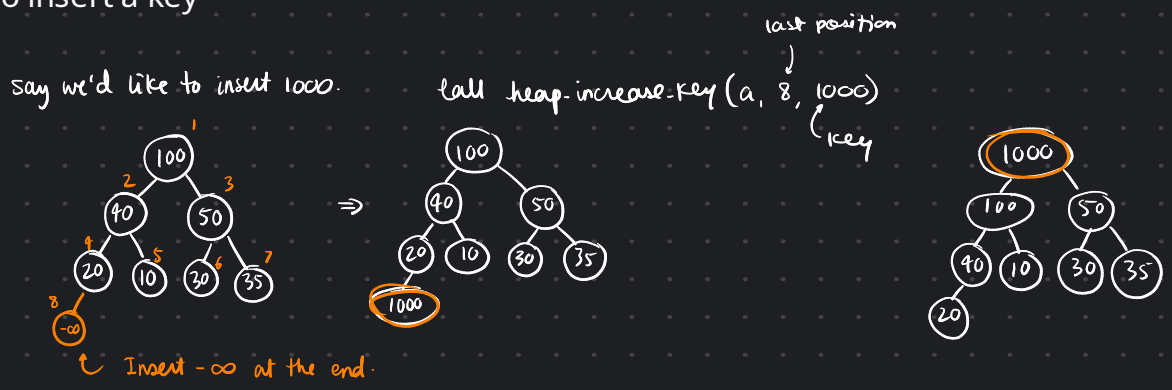
haven't reached root yet

parent < child

swap(parent, child)

Time complexity = O(logn)
Space complexity = O(1)

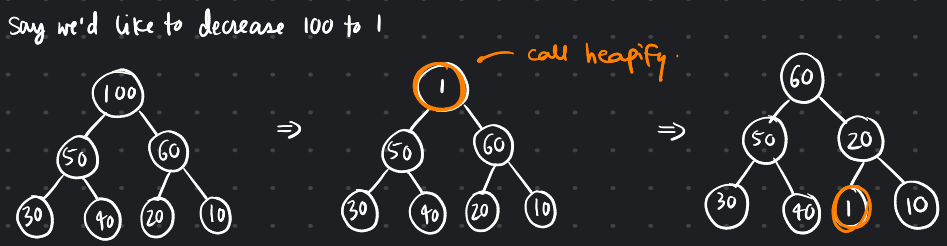
To insert a key




```python
def max_heap_insert(a, key):
    a.heap_size += 1
    a[a.heap_size] = float('-inf')
    heap_increase_key(a, a.heap_size, key)
```

Time complexity = O(logn)
Space complexity = O(1)

To decrease a key



Deleting a key

Find the key: O(n).
Replace with last node and call max_heapify: O(logn).

⇒ T(n) = O(n)

find_max: O(1)
extract_node: O(logn)
insert: O(logn)
increase_key: O(logn)
decrease_key: O(logn)
find_min: O(n)
search: O(n) ← have to do a linear search
delete: O(n)

Heap sort

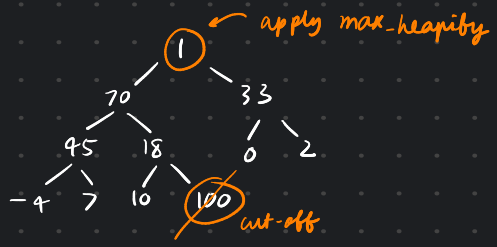
18 45 33 7 70 0 2 -4 100 10 1

Firstly, we'll build a max_heap (min heap for decreasing order).

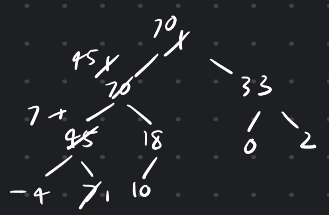


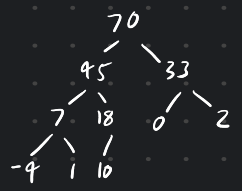
The array comes out as:

100 70 33 45 18 0 2 -4 7 10 1



swap first and last elements,
decrease the heap size
(cut-off the last (max) element)





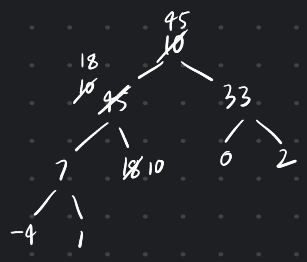
1 70 33 45 18 0 2 -4 7 10 | 100

70 45 33 7 18 0 2 -4 1 10 | 100

exchange these and do the same

In every iteration, the current max element is moved to the end of the current heap and cut-off.

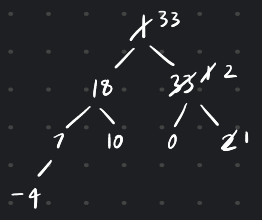
10 45 33 7 18 0 2 -4 1 | 70 100



45 18 33 7 10 0 2 -4 1 | 70 100

swap

1 18 33 7 10 0 2 -4 | 45 70 100



Every time, we apply max_heapify to the root. max_heapify takes O(logn) time, but the size decreases each time. O(h)

$O(\log n) + O(\log(n-1)) + \dots + O(\log(1))$

The logn's dominate

$T(n) \le O(n \log n)$

At 1 time, 1 max_heapify is called. Space = O(logn)

```python
def heap_sort(a):
    build_max_heap(a)
    for i in range(a.length, 1, -1):
        a[1], a[i] = a[i], a[1]
        a.heap_size -= 1
        max_heapify(a, 1)
```

Time complexity:

Best, worst, average cases: $O(nlogn)$

Space complexity: $O(logn)$

In-place: yes

Stable: no

HEAPSORT(A)

1 BUILD-MAX-HEAP(A)
2 **for** $i = A.length$ **downto** 2
3 exchange $A[1]$ with $A[i]$
4 $A.heap\text{-}size = A.heap\text{-}size - 1$
5 MAX-HEAPIFY$(A, 1)$

8.3.5 Binary Heap: GATE CSE 2003 | Question: 23

In a min-heap with n elements with the smallest element at the root, the 7th smallest element can be found in time

A. $\Theta(n \log n)$
B. $\Theta(n)$
C. $\Theta(\log n)$
D. $\Theta(1)$

gatecse-2003 data-structures binary-heap

To get min element, we'll use extract_min 6 times, root will then be 7th min.
To add those elements back, $6 * \log n$ as well. $(6 * \log n)$ $O(\log n)$.

8.3.8 Binary Heap: GATE CSE 2006 | Question: 10

In a binary max heap containing n numbers, the smallest element can be found in time

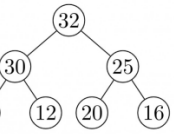
A. $O(n)$
B. $O(\log n)$
C. $O(\log \log n)$
D. $O(1)$

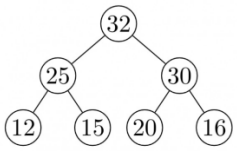
gatecse-2006 data-structures binary-heap easy

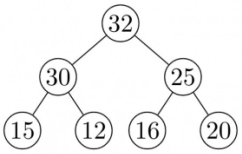
of leaves = $O(\frac{n}{2}) = O(n)$ ∴ linear search on those : $O(n)$.

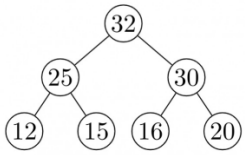
8.3.6 Binary Heap: GATE CSE 2004 | Question: 37

The elements $32, 15, 20, 30, 12, 25, 16,$ are inserted one by one in the given order into a maxHeap. The resultant maxHeap is

A.


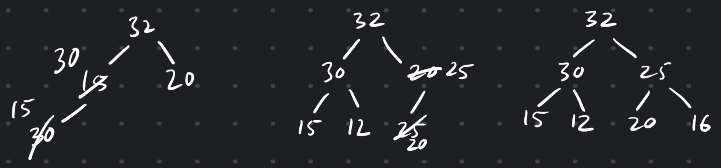
B.


C.


D.


gatecse-2004 data-structures binary-heap easy

This is not build_max_heap. We are inserting elements repeatedly.



8.3.10 Binary Heap: GATE CSE 2006 | Question: 77

Statement for Linked Answer Questions 76 & 77:

A 3-ary max heap is like a binary max heap, but instead of 2 children, nodes have 3 children. A 3-ary heap can be represented by an array as follows: The root is stored in the first location, $a[0]$, nodes in the next level, from left to right, is stored from $a[1]$ to $a[3]$. The nodes from the second level of the tree from left to right are stored from $a[4]$ location onward. An item x can be inserted into a 3-ary heap containing n items by placing x in the location $a[n]$ and pushing it up the tree to satisfy the heap property.

76. Which one of the following is a valid sequence of elements in an array representing $3-$ary max heap?

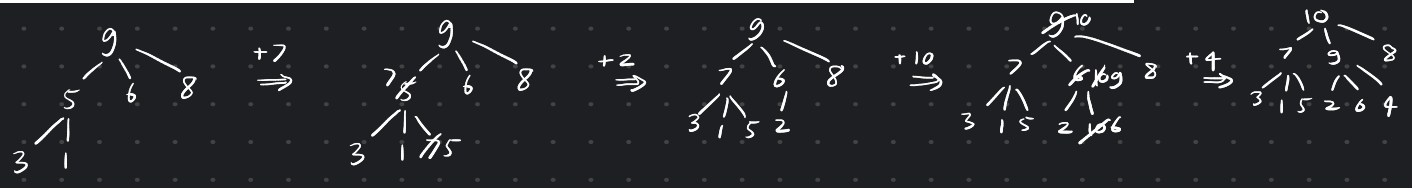
A. $1,3,5,6,8,9$
B. $9,6,3,1,8,5$
C. $9,3,6,8,5,1$
D. $9,5,6,8,3,1$



77. Suppose the elements $7, 2, 10$ and 4 are inserted, in that order, into the valid 3-ary max heap found in the previous question, Q.76. Which one of the following is the sequence of items in the array representing the resultant heap?

A. $10,7,9,8,3,1,5,2,6,4$
B. $10,9,8,7,6,5,4,3,2,1$
C. $10,9,4,5,7,6,8,2,1,3$
D. $10,8,6,9,7,2,3,4,1,5$

gatecse-2006 data-structures binary-heap normal



8.3.11 Binary Heap: GATE CSE 2007 | Question: 47

Consider the process of inserting an element into a $Max\ Heap$, where the $Max\ Heap$ is represented by an $array$. Suppose we perform a binary search on the path from the new leaf to the root to find the position for the newly inserted element, the number of $comparisons$ performed is:

A. $\Theta(\log_2 n)$
B. $\Theta(\log_2 \log_2 n)$
C. $\Theta(n)$
D. $\Theta(n \log_2 n)$

gatecse-2007 data-structures binary-heap normal

New leaf to root has $O(\log n)$ elements, so binary search on that would take $O(\lg \lg n)$.

1.28.17 Time Complexity: GATE CSE 2008 | Question: 47

We have a binary heap on n elements and wish to insert n more elements (not necessarily one after another) into this heap. The total time required for this is

A. $\Theta(\log n)$
B. $\Theta(n)$
C. $\Theta(n \log n)$
D. $\Theta(n^2)$

gatecse-2008 algorithms time-complexity normal

build_heap for $2n$ elements takes $O(2n)$ time $= O(n)$.

(If it were one after the other, it'd be $n \log n$)